
Credit Risk Assessment System

Cloud-Native • MLOps • Microservices • End-to-End ML Platform

1. Project Overview

The **Credit Risk Assessment System** is a **cloud-native, microservices-based machine learning platform** designed to evaluate the creditworthiness of loan applicants in **real time**.

The system predicts loan default risk using ML models such as:

- Logistic Regression
- Random Forest
- XGBoost

Based on prediction scores, applicants are categorized into:

- ☐ **Low Risk** → Loan Approved
- ☐ **Medium Risk** → Manual Review Required
- ☒ **High Risk** → Loan Rejected

This system implements a **full MLOps lifecycle**, including:

- Automated training pipelines
- Model versioning
- Continuous deployment
- Monitoring & drift detection
- Scalable cloud deployment

It runs seamlessly in:

-  Local environment (Docker Compose)
 -  Cloud environment (Kubernetes - AKS / EKS)
-

2. Objectives

The main objectives of this system are:

- Automate credit risk evaluation

- Reduce manual loan approval workload
 - Improve financial risk management accuracy
 - Enable real-time risk scoring APIs
 - Build scalable cloud-native ML infrastructure
 - Implement complete MLOps lifecycle automation
-

□ 3. System Architecture

The system follows a **microservices + cloud-native architecture**.

◆ Core Components

- React Frontend (User Interface)
 - FastAPI API Gateway
 - Authentication Service (JWT / OAuth2)
 - Feature Engineering Service
 - ML Inference Service
 - PostgreSQL Database
 - Redis Cache Layer
 - MLflow Model Registry
 - Monitoring Stack (Prometheus, Grafana, Evidently AI)
 - Kubernetes (AKS / EKS)
-

4. Architecture Diagram

The system architecture includes:

□ User Layer

- React-based UI for loan application and risk visualization

⚙️ Microservices Layer

- API Gateway (FastAPI)
- Authentication Service
- Feature Engineering Service
- Model Inference Service

Data Layer

- PostgreSQL (structured data)
- Redis (caching)

- Object Storage (datasets & artifacts)

☐ MLOps Layer

- DVC (Data versioning)
- MLflow (experiment tracking + registry)
- Training pipelines
- Model validation

🚀 Deployment Layer

- Docker containerization
- Kubernetes orchestration
- CI/CD via GitHub Actions

📊 Monitoring Layer

- Prometheus (metrics)
 - Grafana (dashboards)
 - Evidently AI (data drift detection)
-

🔄 5. End-to-End Workflow

Step 1: Data Ingestion

- Loan applications collected from users
- External credit bureau & banking data integrated

Step 2: Data Processing

- Missing value handling
- Outlier treatment
- Categorical encoding
- Feature scaling

Step 3: Feature Engineering

- Debt-to-income ratio
- Credit utilization ratio
- Employment stability index
- Loan-to-income ratio

Step 4: Model Training

- Logistic Regression
- Random Forest
- XGBoost
- Model selection based on **ROC-AUC score**

Step 5: Experiment Tracking

- MLflow logs:
 - parameters
 - metrics
 - artifacts

Step 6: Model Registry

- Model versioning:
 - Staging
 - Production
 - Archived

Step 7: Deployment

- Docker containerization
- Kubernetes deployment
- FastAPI inference endpoints

Step 8: Monitoring

- Data drift detection
- Model performance tracking
- API latency monitoring
- Alerting system

□ 6. Microservices Design

◆ API Gateway (FastAPI)

- Request routing
- Authentication
- Rate limiting
- Logging

◆ Feature Engineering Service

- Data preprocessing
- Feature transformation pipelines
- Scikit-learn integration

◆ **Model Service**

- Loads ML model from MLflow
- Performs real-time inference
- Returns risk score

◆ **Authentication Service**

- JWT-based authentication
 - Role-based access control (RBAC)
-

7. Data Architecture

Database: PostgreSQL

Tables:

- applicants
- credit_history
- predictions
- model_logs

Cache: Redis

- Fast prediction caching
- Session management

Object Storage

- Training datasets
 - Model artifacts
 - Logs
-

8. MLOps Pipeline

Tools Used

- MLflow → Experiment tracking & model registry
- DVC → Data versioning
- Airflow / Azure ML → Pipeline orchestration
- Evidently AI → Data drift detection

Pipeline Flow

Data → Validation → Feature Engineering → Training → Evaluation →
Registration → Deployment

9. CI/CD Pipeline

Tools

- GitHub Actions
- Docker
- Kubernetes (AKS/EKS)

Workflow

Code Push → Tests → Docker Build → Image Push → Kubernetes Deployment

10. Deployment Strategy

Local Deployment

Using Docker Compose:

- React Frontend
- FastAPI Backend
- MLflow Server
- PostgreSQL Database

Cloud Deployment

Using Kubernetes:

- Azure Kubernetes Service (AKS) / AWS EKS
 - Azure Container Registry (ACR)
 - Auto-scaling enabled
 - Managed PostgreSQL
-

11. Monitoring & Observability

Tools

- Prometheus → Metrics collection
- Grafana → Dashboards
- Evidently AI → Drift detection

Monitored Metrics

- Prediction accuracy
 - API latency
 - Data drift
 - System health
-

12. Security

- JWT Authentication
 - HTTPS (TLS encryption)
 - Role-based access control (RBAC)
 - Secrets stored in:
 - Kubernetes Secrets
 - Azure Key Vault
-

13. Tech Stack

Layer	Technology
Frontend	React.js
Backend	FastAPI
ML	Scikit-learn, XGBoost
MLOps	MLflow, DVC
Pipeline	Airflow / Azure ML
Database	PostgreSQL
Cache	Redis
Containerization	Docker
Orchestration	Kubernetes
CI/CD	GitHub Actions
Monitoring	Prometheus, Grafana

Layer	Technology
Drift Detection	Evidently AI
Cloud	Azure / AWS

14. Key Features

- Real-time credit risk scoring
 - Cloud-native microservices architecture
 - Full MLOps automation pipeline
 - Model versioning & registry
 - Data drift detection
 - Kubernetes auto-scaling deployment
 - Local + cloud hybrid compatibility
 - Secure API-based system
-

15. Business Impact

-  Reduces loan processing time by **70%**
 -  Improves decision consistency
 -  Reduces financial default risk
 -  Enables scalable digital lending systems
 -  Enhances data-driven banking decisions
-

16. Conclusion

This Credit Risk Assessment System demonstrates a **production-grade AI/ML platform** combining:

- Machine Learning
- MLOps
- Microservices Architecture
- Cloud-Native Deployment
- Real-Time Inference Systems

It is designed to be **scalable, secure, and enterprise-ready**, making it suitable for modern financial institutions.
